LLD ENGINEERING PRACTICAL ASSESSMENT

Design a Ride-Sharing Service (Mini-Uber)

A Comprehensive Object-Oriented Design Problem for Python Practitioners

DIFFICULTY	LANGUAGE	KEY FOCUS	TARGET TIME
Intermediate (Post-OOP)	Python 3.10+	SOLID, ABCs, Strategy	90 – 120 Minutes

1. Problem Overview & Learning Objectives

Design a clean, maintainable, in-memory Low-Level Design (LLD) for an on-demand ride-hailing backend engine. This assignment measures your ability to translate ambiguous business rules into cohesive object models while avoiding monolithic god classes and procedural antipatterns.

Encapsulation & State Invariants

Protect state transitions using Pythonic conventions, private attributes, and properties. Prevent invalid mutations (e.g., driver taking two rides).

Polymorphism & Strategy

Extract dispatch algorithms and fare computation strategies behind abstract interfaces using Python's `abc` standard library.

Composition Over Inheritance

Model trips that reference riders, drivers, vehicles, and payment strategies without deep, brittle inheritance hierarchies.

Pythonic Craftsmanship

Proper use of `dataclasses` for immutables, `Enum` for discrete states, standard dunder methods (`__repr__`), and type hints.

2. Core Entities & Attributes

Entity	Required Fields	Domain Responsibilities
Location	`x: float, y: float`	Immutable coordinate value-object. Computes Euclidean distance to another point.
Rider	`id, name, contact, rating`	Initiates ride requests, views ride history, and submits post-ride driver reviews.
Driver	`id, name, vehicle, location, status`	Maintains operational status (`AVAILABLE`, `ON_TRIP`, `OFFLINE`) and updates real-time position.
Vehicle	`reg_no, type, capacity, rate_multiplier`	Base class or hierarchy (e.g., `BikeVehicle`, `SedanVehicle`, `SuvVehicle`).
Trip	`trip_id, rider, driver, pickup, drop, fare, status`	Encapsulates the lifecycle, timeline timestamps, computed route, and final payment resolution.

3. Lifecycle State Transition Chart

Each trip must follow strict state management rules. Raising appropriate custom exceptions (e.g., `InvalidStateTransitionError`) on illegal operations is strictly required:



4. Functional Requirements & System Operations

1. **Rider Registration & Geolocation:** Users and drivers register in the system. Drivers can independently update their `Location(x, y)` and toggle availability status.
2. **Booking Dispatch Request:** A rider requests a ride providing pickup location, destination location, and preferred vehicle tier.
3. **Driver Matching Strategy:** The system matches the most appropriate available driver. Your matching logic must adhere to an abstract base class `DriverMatchingStrategy` so the matching algorithm can be swapped at runtime:
   - *Default Strategy:* Closest physical distance to pickup coordinate.
   - *Alternative Strategy:* Highest-rated available driver within a 5.0 km radius.
4. **Pricing & Fare Calculation Strategy:** Trips compute an upfront estimate and a final settled fare using a pluggable `PricingStrategy`:
   - *Base Pricing:* `(Base Fare + (Distance × Km Rate)) × Vehicle Multiplier`
   - *Surge Pricing:* Introduces a dynamic multiplier (e.g., `1.5x`) during peak load or bad weather conditions.
5. **Trip Lifecycle Execution:** The system provides clear interfaces for `start_trip()`, `complete_trip()`, and `cancel_trip()`.

5. Architecture & Design Patterns Required

Pattern	Applicable Domain Scope	Expected Implementation
Strategy Pattern	Driver Matching & Fare Pricing	Use `abc.ABC` and `@abstractmethod` to define abstract interfaces for swappable algorithms.
State / Lifecycle Guard	Trip State Management	Enforce unidirectional transitions using an `Enum` and explicit check functions.
Manager / Facade	`RideServiceManager`	Single orchestrator coordinating users, active trips, and driver registry without accumulating business rules.

> **Evaluation Checklist (Self-Review Before Submission)**
> Ensure: (1) Zero circular dependencies, (2) No hardcoded math inside the controller, (3) Use of meaningful type hints (`typing.Optional`, `typing.List`), and (4) Proper encapsulation using private attributes where mutation should be protected.

6. Bonus Constraints for Advanced Learners

- **Trip Rating & Driver Averages:** Add bidirectional feedback where post-trip ratings incrementally update the driver's rolling average rating.
- **Cancellation Fee Ledger:** Implement a rule where cancelling a trip in `ACCEPTED` state charges the rider a standard penalty fee credited to the driver.
- **Thread-Safe Emulation (Mock Locks):** Structure driver allocation so two riders booking concurrently cannot be assigned the same driver.